Nested class / Inner class

1. Java Inner Classes
2. Private Inner Class
3. Static Inner Class
4. Access Outer Class from Inner Class

1. Java Inner Classes

In Java, it is also possible to nest classes (a class within a class). The purpose of nested classes is to group classes that belong together, which makes your code more readable and maintainable.

To access the inner class, create an object of the outer class, and then create an object of the inner class:

```
class OuterClass {
        int x = 10;
        class InnerClass {
                int y = 5;
        }
}
public class MyMainClass {
  public static void main(String[] args) {
    OuterClass myOuter = new OuterClass();
    OuterClass.InnerClass myInner = myOuter.new InnerClass();
    System.out.println(myInner.y + myOuter.x);
  }
}
```

In the above program,

5 + 10 = 15

- To access members which is present within the outer class (instance variables, members, inner class) - first create an object for the outer class.

  Ex: **`outer_class ob1=new outer_class`**
  `ob1.members`

- To access members which is present within the inner class (instance variables, members)- create an object for inner class with the help of outer class

  Ex: **outer_class.inner_class ob2=ob1.new inner_class**
  ob2.inner_class memers

- Using ob1 object we can access members of the outer class and we can create object for inner class.
- Using ob2 object we can access members of the inner class.

Private Inner Class

Unlike a "regular" class, an inner class can be private or protected. If you don't want outside objects to access the inner class, declare the class as private:

```
class OuterClass {
  int x = 10;
  private class InnerClass {
    int y = 5;
  }
}
public class MyMainClass {
  public static void main(String[] args) {
      OuterClass myOuter = new OuterClass();
      OuterClass.InnerClass myInner = myOuter.new InnerClass();
      System.out.println(myInner.y + myOuter.x);
  }
}
```

In the above example, private members are nor visible to the outside the class (MyMainClass).

```
        OuterClass.InnerClass myInner = myOuter.new InnerClass();
```

So if you try to access a private inner class from an outside class (MyMainClass), an error occurs:

```
MyMainClass.java:12: error: OuterClass.InnerClass has private access in OuterClass
    OuterClass.InnerClass myInner = myOuter.new InnerClass();
```

Static Inner Class

An inner class can also be `static`, which means that you can access it without creating an object of the outer class:

```
class OuterClass {
  int x = 10;
  static class InnerClass {
    int y = 5;
  }
}
public class MyMainClass {
  public static void main(String[] args) {
    OuterClass.InnerClass myInner = new OuterClass.InnerClass();
    System.out.println(myInner.y);
  }
}
```

Just like `static` attributes and methods, a `static` inner class does not have access to members of the outer class.

Access Outer Class from Inner Class

One advantage of inner classes is that they can access attributes and methods of the outer class:

```
class OuterClass {
  int x = 10;


  class InnerClass {
    public int myInnerMethod() {
      return x;
    }
  }
}
public class MyMainClass {
  public static void main(String args[]) {
    OuterClass myOuter = new OuterClass();
    OuterClass.InnerClass myInner = myOuter.new InnerClass();
    System.out.println(myInner.myInnerMethod());
  }
}
```